

**Week 10 Portfolio Assignment: Implement Dependency Injection in
the Parking System Application**

Master of Science

Information Technology

Object-Oriented Mthd & Pgm II

Gabriel Twizerimana

University of Denver College of Professional Studies

June 7, 2026

Faculty: Nathan Braun, MS

Dean: Bobbie Kite, PhD

Table of Contents

Architecture, Integration, and Dependency Injection Optimization: Final Report for the Parking System Application	1
Introduction	1
System Design	2
High-Level Architectural Layout	3
Structural Dependency Wiring with Google Guice	3
Structural Class Diagram	4
Sequence Diagram: Concurrent Command Processing Workflow	5
Lessons Learned	6
Reflections on Improvements	7
Historical Challenges and Resolutions	7
Retrospective Alterations.....	8
Future Enhancements.....	9
Conclusion	10
References	11

Table of Figures

Figure 1: high-level design of our system.	2
Figure 2: Class Diagram	4
Figure 3: Sequence Diagram	5

Architecture, Integration, and Dependency Injection Optimization: Final Report for the Distributed Parking System Application

Introduction

Modern software engineering depends heavily on designing systems that remain loose, modular, and maintainable as they scale. This paper presents the final engineering report for the Distributed Parking System Application, a network-driven system built using Java to handle customer registrations and vehicle permitting operations across distributed client-server endpoints.

The primary goal of this system is to handle concurrent operational commands efficiently while protecting data integrity across shared states. Throughout its development, the application progressed from a simple, single-threaded object model into a high-performance concurrent server utilizing custom network serialization protocols.

In this final development milestone, the application was refactored to support **Inversion of Control (IoC)** via the Google Guice Dependency Injection framework. By removing hardcoded constructors and static instantiation loops, the application achieves a clean separation of concerns. This report details the system's design, analyzes architectural transitions, reviews technical hurdles, and outlines future enhancements for scaling the platform.

System Design

High-level design diagram for the system based on the specifications of the parking application architecture. This integrated view visualizes how the different tiers (Client, Network, Command, and Domain Registry) connect, highlighting the use of Google Guice for dependency injection, asynchronous task processing, and thread-safe domain state management.

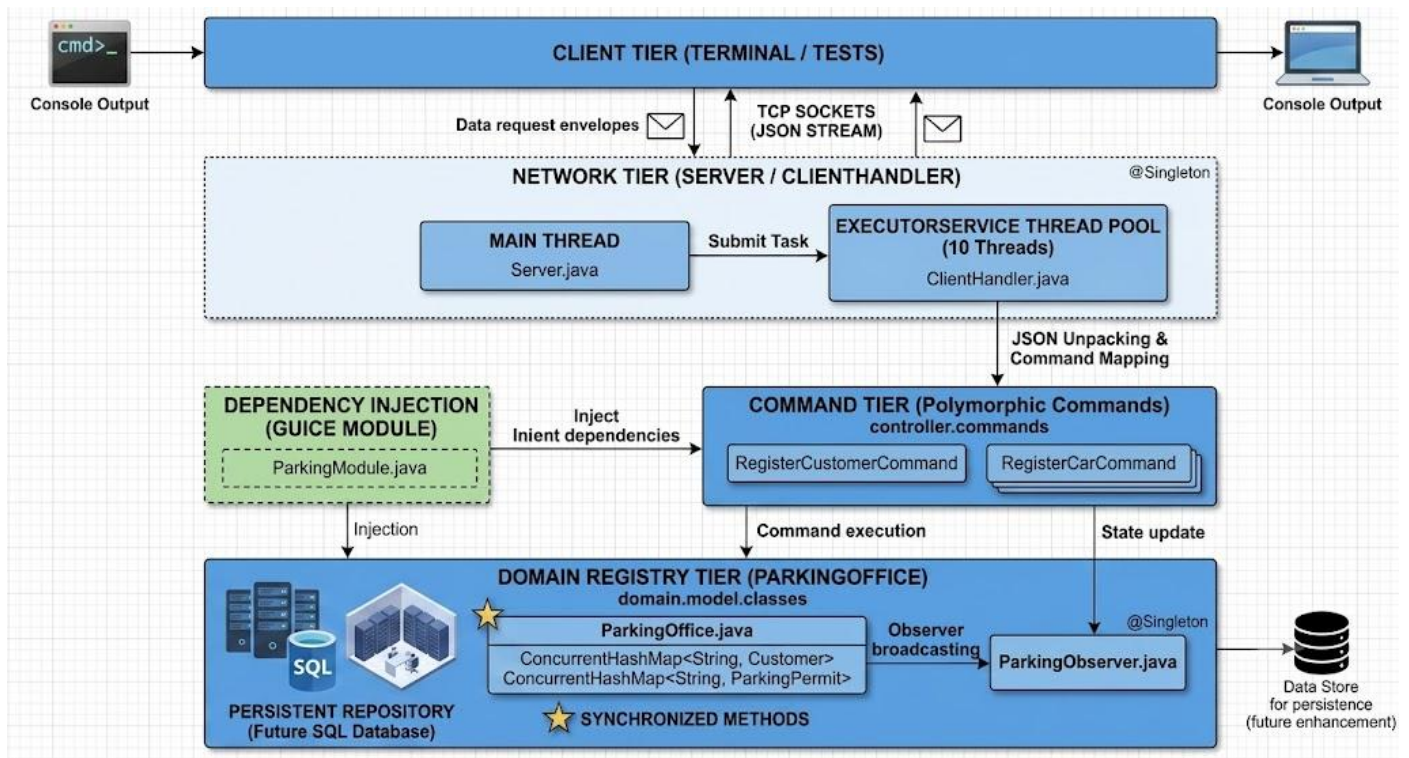


Figure 1: high-level design of our system.

High-Level Architectural Layout

The application follows a clean, layered architectural pattern, dividing responsibilities into distinct packages:

1. **The Shared Communications Layer (shared):** Defines transport protocols (`ParkingRequest` and `ParkingResponse`) that carry serialized data across network sockets.
2. **The Command Processing Layer (controller.commands):** Implements an object-oriented Command Pattern. It converts business actions into executable objects that encapsulate all necessary transaction variables.
3. **The Core Domain Model Layer (domain.model.classes):** Manages the core business data structures (`Customer`, `Car`, `ParkingPermit`, and `ParkingOffice`).
4. **The Network Infrastructure Layer (server and clients):** Handles raw TCP connection pooling, client-server streams, and thread task scheduling.

Structural Dependency Wiring with Google Guice

Integrating Google Guice introduces a central configuration tier via `ParkingModule`. Instead of components managing their own internal dependencies, dependencies are safely declared in constructors using the `@Inject` annotation. The framework then resolves and injects these dependencies automatically at runtime.

Structural Class Diagram

The class diagram below illustrates the structural relationships and dependency injection linkages across the system:

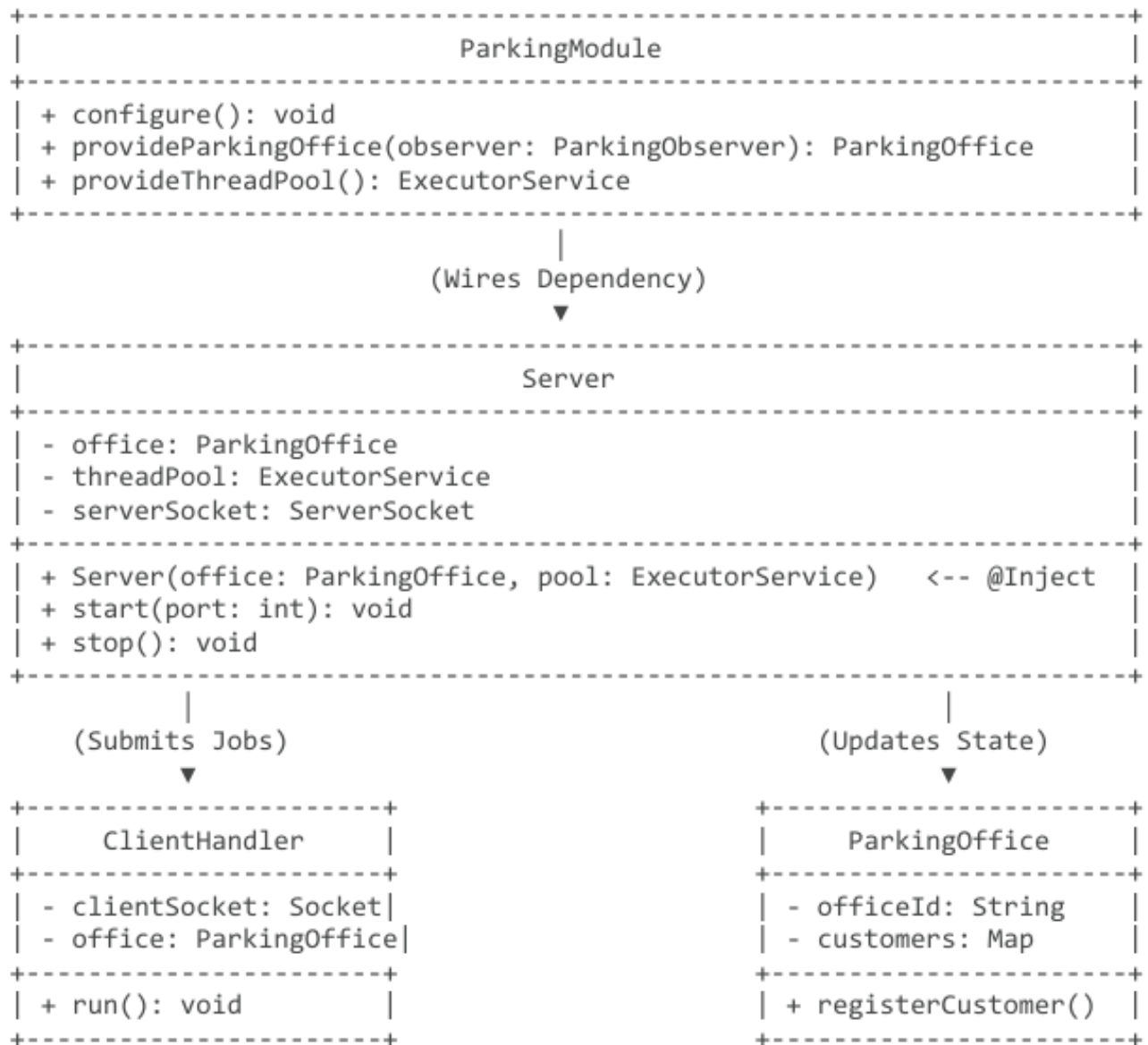


Figure 2: Class Diagram

Sequence Diagram: Concurrent Command Processing Workflow

The sequence diagram below displays the end-to-end communication lifecycle, showing how an incoming request is assigned to a thread pool worker and executed securely using Guice-injected resources:

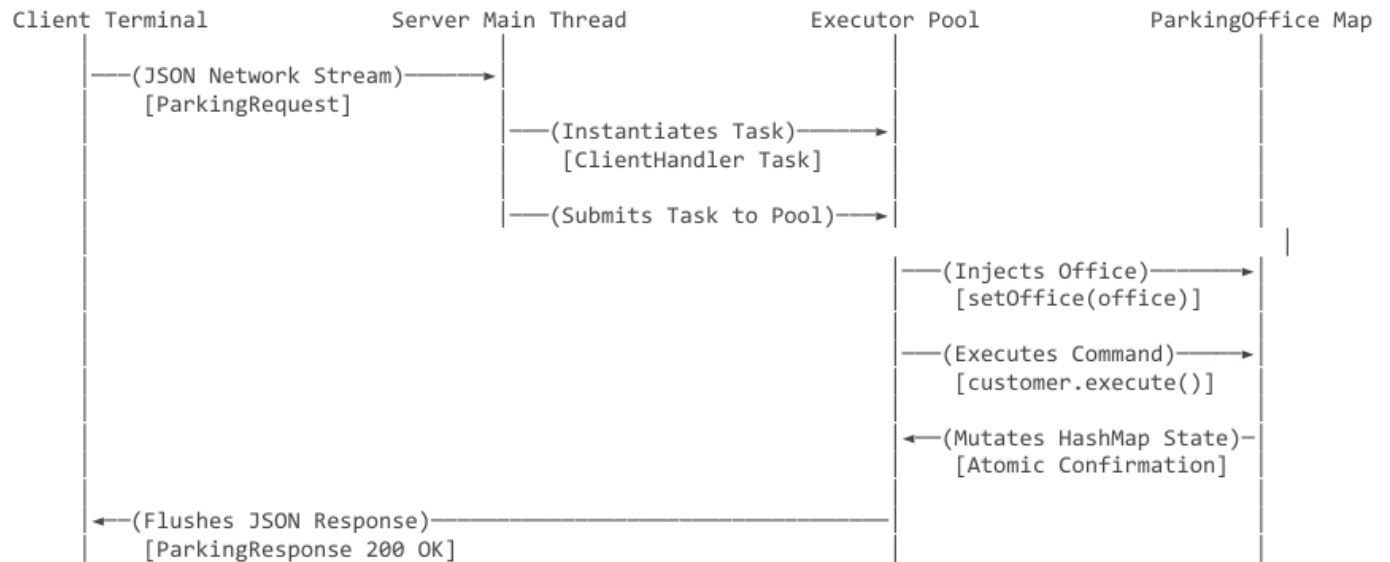


Figure 3: Sequence Diagram

Lessons Learned

Refactoring this system across multiple development milestones introduced several new software design frameworks and patterns:

1. Inversion of Control via Google Guice

Prior to using Google Guice, the Server class had to manually manage instance configurations, which required tightly coupling constructors together. Implementing `AbstractModule` and leveraging `@Provides` annotations demonstrated how outsourcing object creation simplifies system dependencies. It highlights how separating object configuration from execution logic makes a codebase much easier to maintain.

2. High-Performance Socket I/O and Native Diagnostic Tracking

Moving beyond single-client execution required establishing a multithreaded network architecture. Extracting connection processing loops into a standalone Runnable worker pool illustrated the power of parallel processing. Additionally, adding high-resolution telemetry diagnostics via `System.nanoTime()` provided valuable insight into system performance. Watching request processing times drop from several milliseconds down to fractions of a millisecond showed how much efficiency is gained by optimizing execution pipelines.

3. Native Object Serialization Without Framework Abstractions

Building a custom transport layer using Google's Gson library highlighted the structural constraints of reflection frameworks. Encountering JVM validation warnings when mutating final fields clarified how reflection-based deserialization bypasses standard constructors. Learning to manage these constraints by removing final modifiers and providing explicit no-argument constructors was a key breakthrough in designing clean data-interchange layers.

Reflections on Improvements

Historical Challenges and Resolutions

The development journey brought forward several critical engineering challenges that required systemic refactoring:

- **The Stateful Context Leakage Conflict:** During the transition to a command-driven architecture, commands required direct access to `ParkingOffice` to execute business rules. However, serializing the `ParkingOffice` instance over network sockets caused runtime loops and data leaks. This boundary conflict was resolved by marking the repository reference with the transient keyword, which excludes it from serialization. We then added a post-deserialization re-hydration step (`setOffice`) on the server to re-inject the necessary execution context safely.
- **Race Conditions and Memory Corruption:** Introducing a fixed thread pool meant multiple worker threads could modify the parking registries at the same time. Using a standard `HashMap` caused thread safety bugs and corrupted internal data pointers. This was resolved by upgrading the data layer to use `ConcurrentHashMap` and synchronizing mutable registration methods, ensuring thread-safe atomic updates across shared memory spaces.
- **Active File Locks in Windows Environments:** During deployment, running `mvn clean install` repeatedly threw errors because background Java processes held active file locks on dependency JAR files, like `gson-2.11.0.jar`. This taught us the importance of properly managing socket cleanups and shutting down the server application before rebuilding the project.

Retrospective Alterations

Looking back at the architectural choices made throughout the project, there are a few things I would implement differently today:

1. **Early Thread-Safety Design:** I wish I had designed the system with thread-safe data structures from day one. Post-processing class definitions to add thread safety requires careful auditing of mutable methods. Designing with concurrency in mind from the start reduces code revisions and prevents data race bugs early on.
2. **Unified Configuration Management:** Instead of hardcoding properties like default port numbers (12345) or thread pool scaling limits (10), I would use a centralized external configuration file (like `application.properties`). This would allow system properties to be updated dynamically across environments without requiring code re-compilation.

Future Enhancements

If given more development time and resources, the system could be enhanced in three major areas to scale effectively:

1. Data Persistence Tier Integration

Currently, all registered customer records and vehicle permits live purely in-memory inside a volatile heap space, meaning all data is lost whenever the server restarts. A crucial future upgrade would be replacing this volatile collection tier with a permanent data persistence layer, such as an SQL database or a key-value store. This would involve using Guice to inject a persistent repository interface, ensuring data safely survives system restarts.

2. Transport Security and Encryption

The current client-server model passes serialized JSON payloads over raw, unencrypted TCP streams. In a real-world system handling sensitive customer names and payment details, this poses a security risk. Upgrading the network layer to use Secure Sockets Layer (SSL) or Transport Layer Security (TLS) via Java's SSLSocketFactory would encrypt data in transit, protecting the application against data interception.

3. Dynamic Horizontal Scaling and Load Balancing

The fixed thread pool model handles traffic bursts well, but it is ultimately limited by the hardware resources of a single server machine. To scale further, the architecture could be containerized using Docker and deployed across multiple cloud instances. Placing an upstream load balancer (like NGINX) in front of the application would distribute incoming client requests across multiple active servers, allowing the system to scale horizontally to handle thousands of concurrent users.

Conclusion

The development of the Distributed Parking System Application demonstrates how combining solid software patterns with modern development frameworks creates clean, high-performance systems. Moving from tight, hardcoded classes to an asynchronous, command-driven architecture optimized with Google Guice shows the practical value of applying clean design principles.

Overcoming concurrency hurdles, handling serialization constraints, and implementing framework-driven dependency injection has provided deep insight into the lifecycle of enterprise Java applications. The resulting system is clean, modular, and highly performant providing a strong foundation for future scaling and feature enhancements.

References

Bloch, Joshua. 2018. *Effective Java*. 3rd ed. Boston: Addison-Wesley.

Fowler, Martin. 2004. "Inversion of Control Containers and the Dependency Injection Pattern." MartinFowler.com. January 23, 2004. <https://martinfowler.com/articles/injection.html>.

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

Goetz, Brian, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea. 2006. *Java Concurrency in Practice*. Upper Saddle River, NJ: Addison-Wesley.

Harold, Elliott Rusty. 2013. *Java Network Programming*. 4th ed. Sebastopol, CA: O'Reilly Media.

Martin, Robert C. 2018. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Boston: Prentice Hall.

Meszaros, Gerard. 2007. *xUnit Test Patterns: Refactoring Test Code*. Upper Saddle River, NJ: Addison-Wesley.